数字图像处理　第二次 project

04-03、04-04、04-05　　截止/提交日期：2026 年 10 月 14 日

小组分工：

姓名	学号	分工
韩宇	24319050	低通滤波代码、边缘融合及 04-03 报告。
罗云飞	24307120	高通滤波代码、边界对比及 04-04 报告。
李响	24363137	频域相关代码、坐标检查及 04-05 报告。

三人共同检查结果和整理报告，本次对字符图做高斯低通和高通滤波，再用频域相关定位字母 T。输入采用所给的三张图片，计算时保留浮点数，保存图片时再截取到 $[0, 255]$ 并取整。

1　04-03 高斯低通与边缘加权融合

基础实现本部分通过构造二维高斯传递函数完成频域低通。计算过程保留浮点数，以下按实际数据流分析各步代码。

（1）读取灰度图并确定尺寸

```
img = Image.open("input/4.11(a).png").convert("L")
f0 = np.asarray(img)
F = np.fft.fft2(f0)
M, N = F.shape
f = f0.astype(float)
```

`convert("L")` 将图片转为单通道 8 位灰度图；`np.asarray` 得到二维数组，每个元素表示对应像素的灰度。代码先做一次 FFT 再取频谱尺寸。该次频谱仅用于取尺寸，实际滤波函数内会重新变换。转成浮点数再计算以避免原图整数类型在梯度、差值或融合中限制数值范围。

（2）指定高斯函数的大小、中心并生成坐标

```
def gaussian_lowpass_filter(M, N, D0, center=None):
    if center is None:
        center = (M//2, N//2)
    u0, v0 = center
    u = np.arange(M)[:, None]
    v = np.arange(N)[None, :]
```

M, N 控制滤波器尺寸，`center` 指定中心，`u` 的形状为 $M \times 1$，`v` 为 $1 \times N$，分别列出行、列索引。NumPy 广播将二者组合成完整的二维坐标网格，默认中心取 $(u_0, v_0) = (\lfloor M/2 \rfloor, \lfloor N/2 \rfloor) = (274, 279)$，与 `fftshift` 后的零频索引一致。接口允许改变中心，这个实验的普通低通还是以零频为中心。

（3）按到中心的距离计算高斯权重

```
    D_2 = (u - u0)**2 + (v - v0)**2
    exponent = -1 * D_2 / (2.0 * D0**2)
    return np.exp(exponent)
```

`D_2` 是各频率位置到中心的距离平方，由广播得到 $M \times N$ 数组；然后直接使用平方距离省略先开方再平方的过程。返回滤波器：

$$H_L(u, v; D_0) = \exp\left[-\frac{(u-u_0)^2 + (v-v_0)^2}{2D_0^2}\right].$$

D_0 控制响应宽度，减小 D_0 会更强地衰减非零频率，使图像更平滑；增大则保留更多细节。

（4）计算二维频谱并对齐零频

```
def freq_filter(f, H):
    F = np.fft.fftshift(np.fft.fft2(f))
```

`fft2` 将图像分解为二维频率成分，得到包含幅度与相位的复数系数。未中心化时，零频位于数组左上角；`fftshift` 将其移到中心。

（5）逐元素衰减频率成分

```
G = F * H
```

`*` 将对应位置相乘。高斯权重为正实数，乘法衰减对应成分的幅度并保持相位。默认零频处权重为 1。

这里发现当低频保留较多，较高频率受到更强抑制，细碎的纹理和锐利过渡随之减弱；但边缘、细线及噪声都可能包含高频，全局高斯权重无法只凭频率区分它们，导致对一些非噪声、对识别图像有利的边缘也被模糊，后面考虑边缘加权处理的创新尝试，

（6）逆变换并恢复实值图像

```
return np.real(np.fft.ifft2(np.fft.ifftshift(G)))
```

先 `ifftshift` 撤销中心化，再用 `ifft2` 重建图像。`np.real` 去掉浮点运算可能残留的微小虚部。

（7）调用实际参数并保存结果

```
H2 = gaussian_lowpass_filter(M, N, D0=40)
g_low = freq_filter(f, H2)
Image.fromarray(np.clip(g_low, 0, 255).astype(np.uint8)).save("output/lowPass.png")
```

以上为当前普通低通的调用。`np.clip` 将数值限制在 $[0, 255]$，`astype` 转换为 `np.uint8` 时截去小数，再由 Pillow 将数组转成图片。后面的加权方案另用 $D_0 = 10$ 生成更强的低通图片。

改进思路 普通高斯低通在抑制细碎纹理时，也会混合黑白边界两侧的灰度，导致轮廓变宽。所以为了保留明显的边界，先生成高斯低通图，再根据原图的局部梯度构造空间权重，将低通图与原图融合。梯度大小及初始权重为

$$m(x,y) = \sqrt{(\partial_x f)^2 + (\partial_y f)^2}, \qquad w_0(x,y) = 1 - \exp\left[-\frac{m(x,y)^2}{2T^2}\right], \quad T = 10.$$

调用 `np.gradient` 计算梯度，平坦处 w_0 接近 0，主要使用低通结果；明显边缘处 w_0 较大，保留更多原图。但是发现这样仅保护梯度较大的窄边缘时，高斯模糊仍会影响其两侧，所以又进一步扩大保护范围。先将权重四舍五入为 8 位灰度，再通过 `MaxFilter(size=15)` 取 15×15 邻域最大值，最后除以 255，得到 $w \in [0, 1]$。最后使边缘保护向周围扩展约 7 像素。

最终输出为

$$g_{\text{weighted}} = (1 - w)g_L(10) + wf = g_L(10) + w\big[f - g_L(10)\big].$$

这里的 w 按空间位置选择保留程度，最终结果变为高斯低通后再在空间的融合

参数与对比 当前普通低通对照使用 $D_0 = 40$，加权方案的基础低通使用更强的 $D_0 = 10$。选择不同参数是因为融合会重新引入部分原图细节：如果两者使用同样的基础平滑强度，加权结果可能显得平滑不足。因此先增强基础平滑，再利用权重保护边界，在区域内部平滑与轮廓保留之间取得视觉折中。当前配置如下：

方案	高斯参数	空间权重
普通低通	$D_0 = 40$	无
边缘加权融合	$D_0 = 10$	$T = 10$，15×15 最大值扩展



原图　　普通低通，$D_0 = 40$　　加权融合，基础 $D_0 = 10$

图 1: 当前输入、普通高斯低通与采用更强基础平滑的边缘加权结果。两种方案使用不同的 D_0，用于比较实际参数配置下的视觉效果。

结果与局限 普通低通图中的细线、字母及方块边缘较柔和；加权结果中的竖线、大字母和小字符轮廓更清楚，部分低梯度背景区域也受到更强的平滑。说明更强的基础低通与边缘权重可以组合使用，而不必通过整体增大 D_0 来保留细节。不过，上方部分黑方块周围仍能看到明暗光晕和内部灰度不均，右侧密集斑点也保留较多。其原因是原始梯度不能区分有效边界与细碎纹理，最大值扩展又会使附近区域共同获得较高保护权重；边界附近重新引入的原图与较强低通结果之间仍可能存在不自然过渡。

本次对比展示的是两套参数配置的视觉折中，没有固定 D_0 单独比较权重作用，也未以干净参考图计算去噪指标，所以也不能算是去噪精度提高，或完全消除了跨边界模糊。基础低通用来完成题目所需的滤波，$D_0 = 40$ 的普通低通与 $D_0 = 10$ 的加权方案也只是来尝试一种新的滤波，而且效果并不是很理想，保留边缘而模糊内部比我想的复杂。

运行输出 普通低通保存为 `output/lowPass.png`，加权结果保存为 `output/weightLowPass.png`

2 04-04 原图减低通图

将原图减去上一题的低通结果：

$$g_H = f - g_L = \mathcal{F}^{-1}\{(1 - H_L)\mathcal{F}(f)\}.$$

相减使用浮点结果，差图中的负值在保存时置为 0。三组结果见图 2。

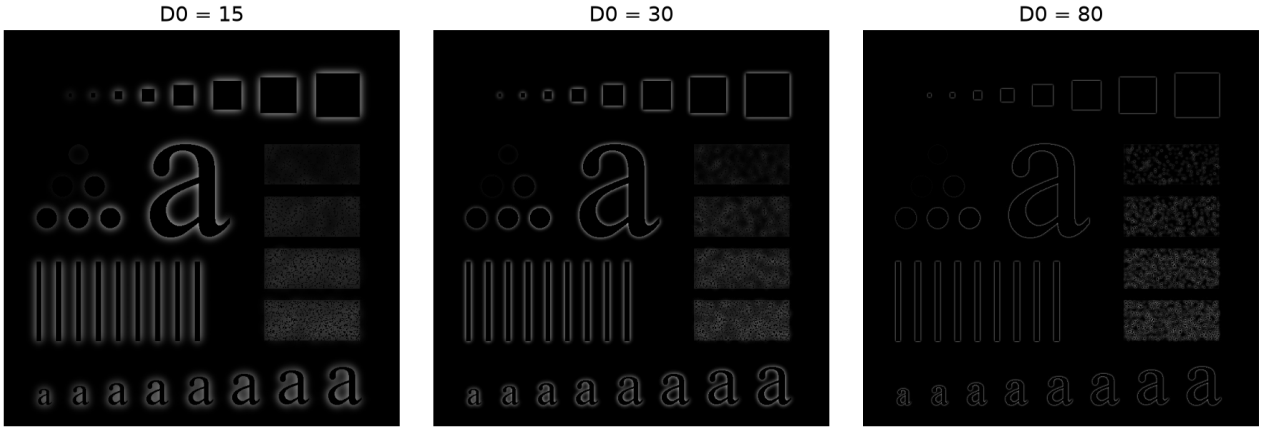


图 2: $D_0 = 15, 30, 80$ 时的高通结果，显示范围均为 $[0, 255]$。

D_0	15	30	80
差图标准差（裁剪前）	45.39	36.58	27.03
图片平均灰度（裁剪后）	10.79	7.31	4.06

(a) 上一题用了 $D_0 = 15$，因此相减结果对应图 4.26(a) 的参数。图 4.26 各图的 D_0 不同，得到的边缘宽度也不同。同一参数下，原图减低通图与使用 $1 - H_L$ 做频域高通一致，计算误差小于 10^{-10}。

(b) 将 D_0 改为 80，即方差从 225 改为 6400，可得到图 4.26(c) 对应的效果。此时减掉的低频成分更多，边缘变细，图像也更暗。

另外比较了边界处理。将图像补零到 1376×1376，D_0 同时改为 30，保持与原尺寸 $D_0 = 15$ 相同的滤波尺度，再截回原图大小。

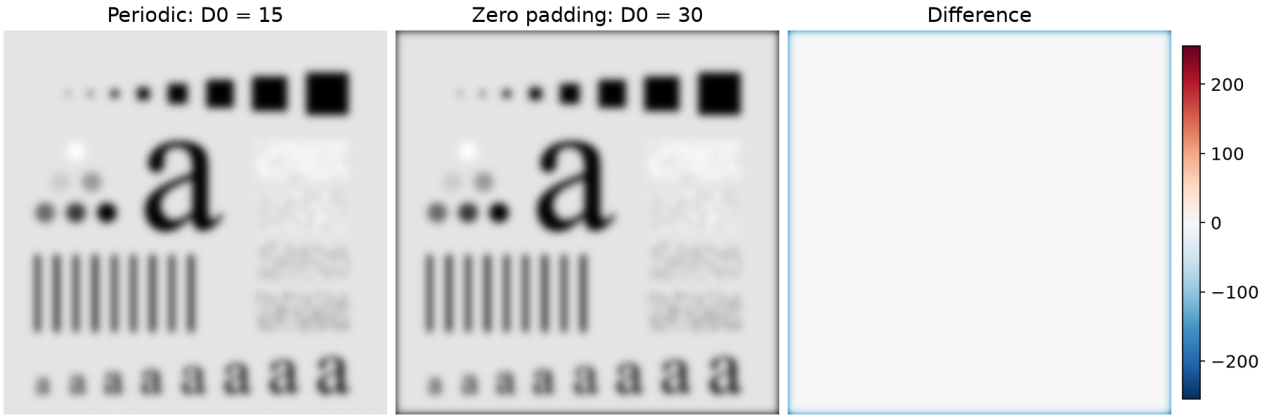


图 3: 不补零、补零后裁剪以及两者的差值。

图 3 中补零后的图像边缘变暗。外侧 64 像素区域的平均绝对差为 11.42，内部差异小于 10^{-10}。这张图的背景较亮，补零在边界引入了黑色区域，影响主要出现在四周。

3 04-05 频域相关

原图为 256×256，模板为 44×49（行 $\times$ 列）。保留模板的黑边，将两图补零到 299×304：

$$P = 256 + 44 - 1 = 299, \qquad Q = 256 + 49 - 1 = 304.$$

取模板频谱的共轭，与原图频谱相乘，再做逆变换：

$$C = \mathcal{F}^{-1}\{\mathcal{F}(f_P)\,\overline{\mathcal{F}(t_P)}\}.$$

在模板完整落入原图的 213×208 个位置中取最大值，结果见图 4。所给模板比教材中的模板大，因此补零大小按实际尺寸计算。

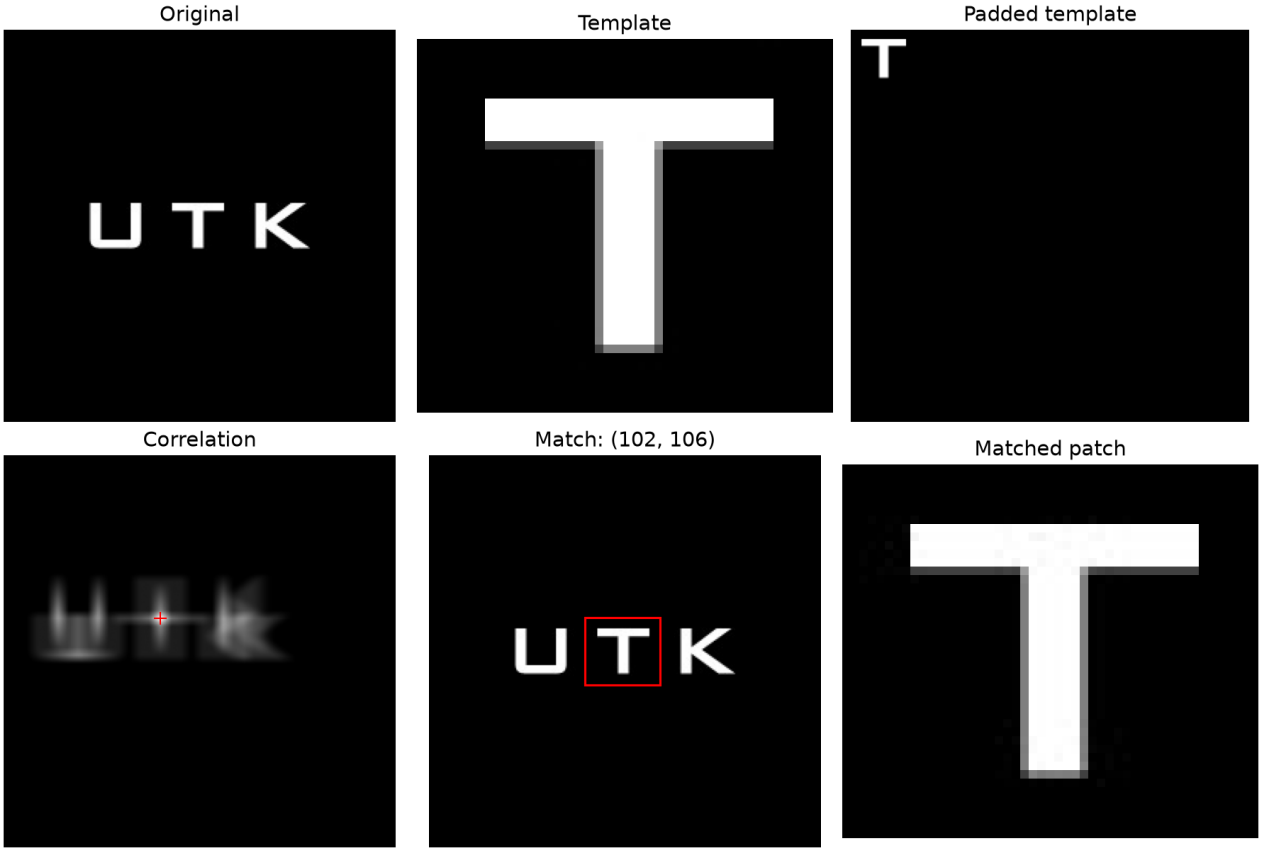


图 4: 原图、模板、补零模板、相关图和匹配结果。

结果	数值
最大值位置 (x, y)，从 0 开始	$(102, 106)$
从 1 开始计数的位置	$(103, 107)$
模板宽、高	$49, 44$
最大相关值	21 352 516

匹配位置是图中的 T。x 为列、y 为行，坐标表示模板左上角。用该位置的图像块与模板直接相乘求和，也得到 21 352 516。相关图只在显示时缩放灰度，计算使用原始相关值。

安装 NumPy、Pillow 和 Matplotlib 后，运行 `python project2.py` 即可生成结果图片。FFT 调用 NumPy 的 `fft2/ifft2`，Pillow 用于读写图片，Matplotlib 用于绘图。

参考：课程题目截图；Gonzalez 与 Woods《数字图像处理》相关章节；DIP Laboratory Projects，04-03 至 04-05。